

Dynamic Programming

INTRODUCCIÓN TEÓRICO-PRÁCTICA AL APRENDIZAJE POR REFUERZO

Profesor: Matias Bilkis

Cuatrimestre: Segundo cuatrimestre 2026

Integrantes

Agustín Calo	390/23	Paz Lavenia	64/23
Rocco Gasparini	102/24	Silvano Picard	637/24
Maria Delfina Kiss	72/23	Ingrid von Foerster	150/23

Análisis de Value Iteration

Para este análisis, partimos del código base de *Value Iteration* que nos dieron en clase. El entorno modelado es un laberinto representado como una grilla de $n \times n$ con un 20 % de paredes ubicadas al azar y un estado objetivo. El objetivo del experimento fue variar el tamaño de la grilla (desde $n = 10$ hasta $n = 100$, en saltos de 10) para evaluar cómo escala el algoritmo.

A partir de las ejecuciones, generamos los gráficos de rendimiento (ver Figura 1) y observamos dos comportamientos bien marcados:

Análisis de Escalabilidad: Value Iteration en Laberintos Aleatorios

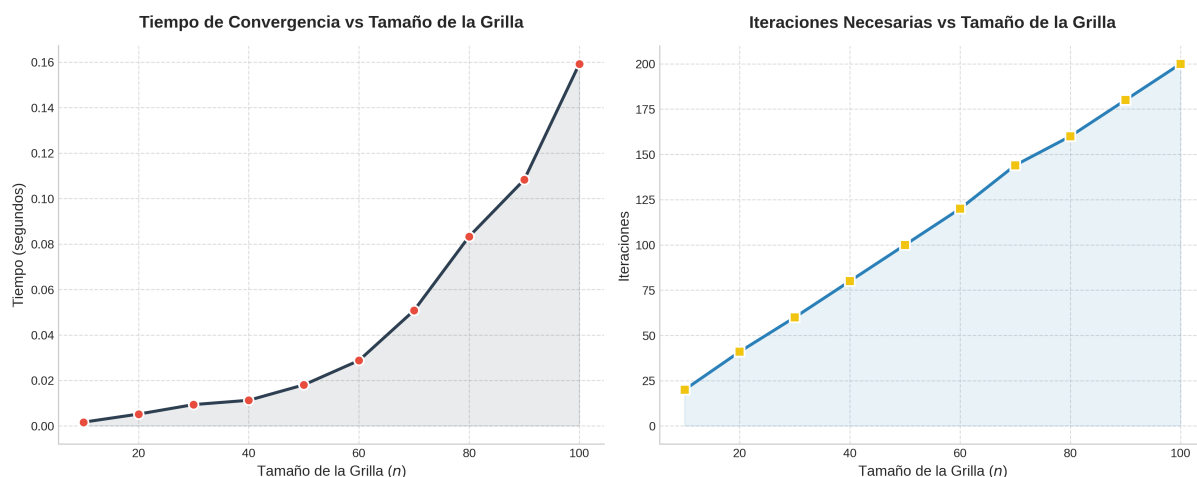


Figura 1: Tiempo de convergencia e iteraciones necesarias en función del tamaño de la grilla.

- **Las iteraciones escalan linealmente (gráfico derecho):** La cantidad de barridos que necesita el algoritmo para converger crece de manera directamente proporcional al tamaño n (va de 20 a 200 iteraciones). Esto tiene sentido ya que la señal del valor de recompensa tiene que propagarse desde la meta hasta el extremo opuesto del laberinto, y esa distancia máxima crece linealmente a medida que la grilla se hace más grande.
- **El tiempo de ejecución se dispara (gráfico izquierdo):** A diferencia de las iteraciones, la curva de tiempo muestra un crecimiento de tipo polinómico. Esto ocurre porque la cantidad total de estados (celdas) crece al cuadrado (n^2). Al multiplicar las iteraciones, que crecen en el orden de $O(n)$, por un procesamiento matricial sobre n^2 celdas, el costo computacional total termina rondando un $O(n^3)$.

En conclusión, los tiempos absolutos son muy bajos (el entorno de 100×100 converge en menos

de 0.15 segundos). Sin embargo, el experimento deja clarísimo cómo afecta la maldición de la dimensionalidad a *Value Iteration*, si siguiéramos aumentando el tamaño del laberinto, el hecho de tener que evaluar todo el espacio de estados en cada iteración haría que el tiempo se vuelva inmanejable rápidamente.